

A chatbotoktól az agentic codingig

Teljes cikk, szerkesztői audit és kiegészítő anyagok - publikációra előkészítve

Tartalom. Ez a csomag a kutatási dosszié (I-VIII.) jóváhagyása utáni fázist tartalmazza: a **IX. teljes cikket** (kb. 2500-3500 szó), a **X. szerkesztői auditot** (nyolc szempont, 1-10 skála), valamint a kiegészítő anyagokat (newsletter-bevezető, LinkedIn-poszt, négy ábra, öt kiemelhető állítás, hat további cikkötlet). A cikk a dosszié VIII. vázlatát követi, a benne szereplő adatok forrása a dosszié Kulcsforrások szakasza.

Paraméter	Érték
Formátum	Hosszú formátumú szakcikk + szerkesztői audit + terjesztési csomag
Hang	CtrlPlane: gyakorlati, technológiailag pontos, józan, hype-mentes
Vezérszempont	Nem a technológiai idővonal, hanem az ember-gép munkamegosztás változása
Terjedelem	A főcikk kb. 2600 szó (a 2500-3500 sávban)
Státusz	A benchmark- és piaci számok a megjelenés előtt újraellenőrizendők (lásd X. audit)

IX. A TELJES CIKK

A chatablaktól a pull requestig

Hogyan tanult meg dolgozni velünk az AI - és miért nem ugyanaz a gyorsabb kód, mint a jobb szoftver

Egy fejlesztő 2022-ben bemásol egy hibás függvényt a ChatGPT-be, elolvassa a választ, és kézzel beilleszti a kódba. Ugyanez a fejlesztő néhány évvel később már csak egy issue-t ad át egy terminálban futó agentnek; a program megkeresi a releváns fájlokat, módosítja a kódot, lefuttatja a teszteket, kijavítja a hibát, és összefoglalja a diffet. A két jelenet között nem pusztán egy gyorsabb modell áll, hanem egy másik munkamegosztás. Ez a cikk azt járja körbe, mi változott meg valójában - nem technológiai idővonalként, hanem annak kérdéseként, hogy ki mit csinál, és hol húzódik a szoftverfejlesztés legkisebb értelmes munkaegysége. A vezérfonal végig ugyanaz: nem a modellek mérete vagy a benchmark-pontszám a lényeg, hanem hogy ki fogalmazza meg a feladatot, ki írja meg a megoldást, és ki vállalja érte a felelősséget.

Előre bocsátva a végkifejletet: a változás lényege nem az, hogy az AI gyorsabban ír kódot, hanem hogy a munka legkisebb egysége fokozatosan elmozdult - a sorról és a függvényről a komplett feladat és az issue felé. Ezzel párhuzamosan az ember szerepe is áttevődik a közvetlen implementációról a specifikáció, az irányítás, az ellenőrzés és a felelősség felé. A cikk ezt az utat járja végig négy állomáson - chatbot, copilot, vibe coding, agentic coding -, majd szembesíti a lelkesedést azzal, amit a kontrollált mérések ténylegesen mutatnak.

1. A chatbot még válaszolt

A klasszikus chatbot nem értette a beszélgetést, hanem előre definiált szándékokat ismert fel. Az ELIZA 1966-ban mintaillesztett, a későbbi ipari rendszerek (Rasa, Dialogflow) intent-felismeréssel és entitáskinyeréssel, állapotgép mentén választottak következő lépést. A felület természetes nyelvű volt, a működés viszont zárt és szabálykött.

A korszak nem volt meddő: a szándékfelismerés és a slot filling számtalan ügyfélszolgálati botot és hangasszisztentst működtetett. A programozáshoz azonban keveset tett hozzá: a logikát, a kódot és a hibakeresést végig az ember vitte, a gép legfeljebb sablonválaszt adott. A valódi töréspont nem a párbeszéd-felületben, hanem a mögötte álló modell általánosságában érkezett: amint egy nyelvi modell tetszőleges szöveget tudott kezelni, a chat alkalmassá vált arra is, hogy kódot magyarázzon, generáljon és átírjon.

A 2022. novemberi ChatGPT ezt törte meg: szabad szöveget kezelt, és a programozás azonnal az egyik legfőbb felhasználási terep lett. Karpathy híressé vált megjegyzése szerint „a legforróbb új programozási nyelv az angol”. A munkamenet azonban változatlan maradt: az ember feltette a kérdést, a választ pedig maga értelmezte, másolta és illesztette be. A chat nem látta a projektet, hallucinált nem létező könyvtárakat és API-kat, és hiányzott belőle a repository-kontextus. A teljes integráció - az illesztés, az adaptálás, a hibakeresés - emberi feladat volt. Ha pedig a kontextus a szűk keresztmetszet, adódik a következő lépés: vigyük az AI-t a kód mellé.

2. A copilot beköltözött az editorba

A külön chatablaktól az AI átköltözött a fejlesztői környezetbe. Az inline kiegészítés, az IDE-be épített chat és a repo-kontextus a 2021-22-es GitHub Copilottal vált tömegessé: az AI ott ajánlott, ahol a fejlesztő éppen dolgozott. Az alapegység továbbra is a kódrészlet volt, és a fejlesztő minden sort maga irányított.

Érdemes látni, mi maradt változatlan. A copilot a fejlesztő döntéseit gyorsította, de nem vette át őket: az ember választotta ki, mit fogad el és mit ír felül, és ő illesztette a kódot a rendszer egészébe. A GitHub Next kutatásai (Kalliamvakou és munkatársai) a befejezési arány és az elégedettség javulását is jelezték, a Stack Overflow 2025-ös felmérése szerint pedig a Copilot a fejlesztők körében továbbra is a legelterjedtebb eszközök között maradt. A kérdés tehát nem az volt, hogy segít-e, hanem hogy meddig terjedhet a segítség.

Ennek a szakasznak van a legisztább kontrollált bizonyítéka. Egy randomizált kísérletben (Peng és munkatársai) a Copilotot használó csoport egy izolált JavaScript-szerver feladatot mintegy 55,8%-kal gyorsabban oldott meg, magasabb befejezési aránnyal. A szám látványos, de a kontextusa döntő: greenfield, önálló, jól körülhatárolt feladatról van szó. Épp ez lesz a cikk visszatérő motívuma - egy zöld mezős feladaton mért gyorsulás nem általánosítható egy érett, nagy kódbázisra. A copilot tehát valódi segítség, de még mindig a fejlesztő a közvetlen kódoló. Ha a kiegészítés ennyire jól megy, jön a következő kérdés: mi történik, ha az egész terméket nyelven írjuk le?

3. A természetes nyelv lett a prototípus nyelve

2025 februárjában Andrej Karpathy egy tweetben adott nevet annak, amit sokan már csináltak: a vibe codingnak. Az eredeti jelentés radikális - „teljesen átadod magad a vibe-nak”, elfogadod és futtatod az AI outputját, a hibát visszaadod, és a kódra mint olyanra nem figyelsz. A böngészős app builderek (Lovable, Bolt, v0, Replit Agent) ezt vitték tömegpiacra: promptból teljes, működő prototípus, sokszor nem-fejlesztők kezében. A piac robbanásszerűen nőtt; a Lovable a jelentések szerint nyolc hónap alatt százmillió dolláros éves bevételi szinthez ért.

A fogalom gyorsan kultúrjelenséggé vált: a Collins szótár 2025-ös „Word of the Year” választása is jelezte, mennyire bevonult a köztudatba. A felfutás mögött valós munkamód állt: nem-fejlesztők - terméktulajdonosok, marketingesek, alapítók - először tudtak működő prototípust előállítani anélkül, hogy egyetlen sort is megírtak volna. Egy ilyen eszköz órák alatt elvisz a demo-szintig, ami korábban heteket vett volna igénybe. A kérdés csak az, hogy ez a prototípus meddig marad fenntartható, ha valódi felhasználók és valódi adatok kerülnek mögé.

Két dolgot azonban érdemes kimondani. Először: Simon Willison éles elhatárolása szerint ha átnézted, tesztelted és megértetted a kódot, az már nem vibe coding, hanem egyszerűen az LLM gépíróként való használata. A köznyelv ezt összefoglalja, és a „vibe coding” ma egyszerre jelöl gyors prototípust, figyelmetlen kódelfogadást, no-code réteget és tapasztalt fejlesztői gyorsítást. Másodszor: a működő demo és a fenntartható rendszer között valós szakadék húzódik. A látványos React-felszín után jön a jogosultságkezelés, az autentikáció és az adatbázis konfigurálása - itt akad el a nem-technikai felhasználó. A Guardio Labs „VibeScamming” néven mutatott be sérülékenységet generált kódban, egy ismertté vált esetben pedig egy AI-agent törölt egy éles adatbázist, miközben explicit utasítás tiltotta volna. A prototípustól tehát a megbízható, ellenőrizhető delegálás felé vezet az út.

4. Már nem kódot, hanem feladatot adunk

Az agentic coding a munka legkisebb egységét mozdtítja el: a kódrészletről a feladatra vagy az issue-ra. Az agent egy cél érdekében több lépést önállóan megtervez, eszközöket használ, fájlokat módosít, parancsokat futtat, és a visszacsatolásból korigálja: cél, feltérképezés, terv, módosítás, teszt, javítás, diff és pull request. A terminál-natív eszközök - Claude Code, OpenAI Codex, Devin - ezt tették mindennapi munkamóddá. A Codex heti többmillió felhasználói kört jelent, az Anthropic pedig azt állítja, hogy belső kódjának többségét már a Claude Code írja.

Addy Osmani megfogalmazásában az „agentic engineering” épp attól válik el a vibe codingtól, hogy fegyelmet visz a folyamatba: specifikáció, tesztelés és review kíséri, nem a vak elfogadás. A különbség tehát nem a modellben van, hanem a munkamódban. A gyakorlatban ez azt jelenti, hogy az agent annyit ér, amennyit a köré épített keret enged: világos feladatleírás, gyors tesztkör és szűk jogosultságok nélkül a legügyesebb modell is kárt okozhat. Ugyanaz az eszköz lehet tehát értékes páros és felügyelet nélküli kockázat - a keret dönt.

Itt fontos a józanság. Armin Ronacher találóan „catnip for programmers”-nek nevezi az agentic codingot - vonzó és addiktív -, de hozzáteszi, hogy az eszközöket védeni kell az „LLM chaos monkey” ellen, vagyis az ellen, hogy a modell teljesen rosszul használja őket. Az agent nem egyetlen modell: működéséhez sandbox, szűk jogosultságok és emberi jóváhagyási pontok kellenek. A kérdés pedig adódik: mit tudnak ezek a rendszerek ténylegesen, a demókon túl?

5. Mit tudnak ténylegesen?

A benchmarkok felső becslések, nem üzemi mércék. A SWE-bench valós GitHub-issue-kon méri, hogy egy modell képes-e végigvinni egy javítást; a 2024-es Verified változat 500 emberi validált feladatán a frontvonal mára gyakorlatilag telítődött, több modell is a 80% körüli sávban tolong. Ez azonban nem azt jelenti, hogy a feladat „megoldott”.

Ezt egy másik mérce is alátámasztja. A régi benchmarkok, mint a HumanEval, rég telítődtek, mert csak izolált függvényeket mérnek, nem valós repository-munkát. A Laude Institute Terminal-Benchje épp ezért a terminál tényleges kezelését teszteli - parancsok futtatását, hibakeresést, többlépéses munkafolyamatokat -, és itt a legjobb eredmények is szerényebbek. A tanulság, hogy a szám mindig csak annyit ér, amennyit a mögötte álló feladat: a szűk, jól definiált példa és a nyílt végű, valós környezet között nagy a távolság.

A kontaminációálló mércék józanító képet adnak. A Scale SEAL SWE-bench Pro változatán, standardizált eszközkészlettel, az eredmények mintegy 45-59%-ra esnek, privát, nem nyilvános kódbázison pedig 15-23%-ra - akár 35 százalékpontos zuhanás a Verifiedhez képest. Nem véletlen, hogy a benchmark-kontamináció (a modellek szó szerint reprodukálták a megoldó patcheket) miatt egyes szereplők 2026 elején abba is hagyták a Verified-eredmények jelentését. A tanulság kettős: egyrészt a mérési szám és a production-teljesítmény között nagy a távolság, másrészt egyre inkább nem a modell, hanem a köré épített harness (eszközkészlet, sandbox, kontextuskezelés) dönti el a tényleges eredményt. És ha gyorsan generál a rendszer, abból még nem következik, hogy jobb szoftver születik.

6. A gyorsabb kód nem feltétlenül jobb szoftver

A legösszetörtebb intuíció az, hogy az AI gyorsabbá tesz, és ez önmagában jó. A METR kontrollált kísérlete mindkét felet éles megvilágításba helyezi. Tapasztalt, nyílt forrású fejlesztők saját, ismerős kódbázisukon 2025 elején nem gyorsultak, hanem mintegy 19%-kal lassultak az AI-eszközöktől - miközben azt hitték, hogy gyorsabbak lettek. Az észlelés és a mért valóság szétvált. (Egy későbbi, késő-2025-ös eszközökkel végzett követő mérés már gyorsulást jelez az eredeti résztvevőknél, de erős szelekciós torzítással - vagyis a kép időben és mintától függően változik.)

A METR eredményét könnyű félreolvasni, ezért fontos a pontosság: nem azt mondja, hogy az AI haszontalan, hanem hogy a tapasztalt fejlesztő ismerős, érett kódbázison másképp viselkedik, mint amit a kontrollálatlan önbevallás sugall. Az észlelés és a mért teljesítmény közötti rés önmagában tanulság: ha nem mérünk, hajlamosak vagyunk túlbecsülni a saját gyorsulásunkat, és ez épp a legrosszabb alapja egy fontos beruházási vagy folyamat-döntésnek.

A minőségi oldal még egyértelműbb. A Veracode 2025-ös vizsgálata szerint a generált kód mintegy 45%-a tartalmazott OWASP Top 10 típusú sérülékenységet, és a biztonsági hibák aránya többszöröse volt az emberi alapvonalnak; az újabb és nagyobb modellek nem teljesítettek jobban. Az Apiiro elemzése a jogosultság-eszkalációs utak és a tervezési hibák jelentős növekedését jelezte, a GitClear longitudinális adatai pedig a refaktor visszaesését, a duplikáció többszöröződését és a kód-churn növekedését. A logika egyszerű: az AI csökkenti a kód előállításának költségét, de növeli az ellenőrizendő kód mennyiségét és a review-terhet. A DORA találó megfogalmazásában az AI „erősítő”: a rendezett szervezetben gyorsít, a kaotikusban a diszfunkciót nagyítja fel. Ha pedig nő a review-teher, mi lesz a fejlesztő szerepével?

7. Mi történik a fejlesztő szerepével?

A hangsúly a közvetlen implementációtól a specifikáció, az architektúra, az ellenőrzés és a felelősség felé tolódik - de juniornál és seniornál másképp. A junior oldalán a számok nyugtalanítóak: a Stanford és az ADP elemzése szerint a 22-25 éves, AI-nak erősen kitett szakmákban dolgozók foglalkoztatása 2022 óta mintegy 13%-kal relatíve csökkent, a tech-gyakornoki hirdetések pedig látványosan visszaestek.

Steve Yegge ellennarratívája szerint épp fordítva is olvasható a helyzet: az AI a kezdőt korábban juttatja érdemi munkához, mert a legrutinszerűbb feladatok egy részét leveszi a válláról. A két értelmezés nem zárja ki egymást: előfordulhat, hogy a belépés küszöbét magasabbra teszi, miközben annak, aki bejut, több eszközt ad a kezébe. Épp ezért szabad csak óvatosan okságot olvasni a munkaerőpiaci számokból.

Itt azonban kötelező az ellenbizonyíték. Humlum és Vestergaard dán adatokon nem talált különbséget az AI-t adoptáló és nem-adoptáló cégek junior-felvétele között, és súlyos makrokonfundálók torzítják a képet: a kamatkörnyezet, a pandémia utáni túlfoglalkoztatás kiigazítása és az informatikai diplomások létszámának évtizedes megduplázódása egyaránt nyom. A korreláció tehát erős, az okság vitatott. A senior oldalon a szerep inkább felértékelődik: az „agent manager” a rendszertervezést, a feladatbontást, az elfogadási kritériumokat, a tesztstratégiát és a review-t viszi. A megtartandó ellenérv viszont az, hogy ha az AI az architektúrában, a hibakeresésben és a tervezésben is javul, akkor a „magasabb szintű gondolkodás” nem feltétlenül tartós emberi menedék. A gyakorlati kérdés ezért nem az, hogy eltűnik-e a fejlesztő, hanem hogy hogyan érdemes ma használni ezeket az eszközöket.

8. Hogyan érdemes ma használni?

A leghasznosabb keret nem eszközökben, hanem feladattípusban és kockázatban gondolkodik. Alacsony kockázatú, jól körülhatárolt munkánál (boilerplate, dokumentáció, egyszerű átalakítás) az agent szabadon engedésének hozadéka magas, a kára alacsony. Közepes kockázatnál (több fájlt érintő feature, migráció) már érdemes szűkebb feladatot és erős tesztet adni. Magas kockázatú területen (autentikáció, pénzügyi logika, kriptó, production deploy) az emberi jóváhagyás nem opcionális.

A keret lényege, hogy nem az eszköz, hanem a feladat kockázata dönt. Ugyanaz a terminál-agent lehet ideális egy dokumentáció-generáláshoz és veszélyes egy fizetési modul átírásához - a különbség nem a modellben, hanem abban van, mennyit bízunk rá felügyelet nélkül. Érdemes ezért a feladatokat először kockázat szerint beosztani, és csak utána választani módot és eszközt; így a gyorsítás ott jelentkezik, ahol olcsó a hiba, és marad az emberi kontroll ott, ahol drága.

A három mód a gyakorlatban egyetlen munkanapon belül is váltakozik. Egy ismeretlen hibánál a fejlesztő először tanácsadóként használja az AI-t, hogy megértse az okot; egy ismerős feature-nél páros programozóként, soronkénti irányítással dolgozik vele; egy jól körülhatárolt, mechanikus feladtnál pedig felügyelt agentként enged neki teret, amely önállóan végigviszi a munkát. A lényeg nem az, hogy végleg eldöntsük, melyik a helyes mód, hanem hogy tudatosan válasszunk a feladat ismeretében.

Három működő mód van, és nem versengenek, hanem feladatonként váltakoznak: az AI mint tanácsadó (chat), mint páros programozó (copilot) és mint felügyelt agent. A DORA „erősítő”-tapasztalata itt is irányadó: a haszon nem magától jön, hanem a szervezet érettségétől függ. Willison észrevétele szerint létezik „AI-kompatibilis kódbázis”: gyors tesztkör, világos struktúra és a kontextusfájlok (CLAUDE.md, AGENTS.md), amelyek ténylegesen segítik az agentet. Két határt azonban érdemes tiszteltetni: a túl széles agent-jogosultság veszélyes, a kontextusmérnökség pedig nem helyettesíti a review-t, csak előkészíti. És ha ma már ennyit bízhatunk az agentre, mi jön ezután?

9. Mi jön ezután?

A 2026-os irány a hosszabb autonóm munkamenetek, a háttérben futó és párhuzamos agentek, a több agent koordinációja, az issue-to-PR pipeline és az automatikus review felé mutat - a fejlesztő egyre inkább „agent manager”. A termékek (Cursor 3 párhuzamos build, Antigravity 2.0, Devin Desktop, Codex desktop) már ezt a munkamódot építik. Közben a frontvonalbeli modellek képessége közel ért egymáshoz, így a verseny egyre inkább a harness és a scaffold szintjére tolódik. Hogy ennek mi a helyes formája, még vitatott: az Anthropic multi-agent kutatórendszere mellett a Cognition kifejezetten amellett érvel, hogy a legtöbb esetben ne építsünk multi-agentet.

A konvergencia gyakorlati következménye, hogy a választásban egyre kevésbé a „melyik modell a legjobb” kérdés számít, és egyre inkább az, hogy melyik eszköz illeszkedik a csapat munkafolyamatához és kódbázisához. A multi-agent vitában sincs kész válasz: a több párhuzamos agent növelheti az átbocsátóképességet, de a koordináció és a review terhét is, így sok esetben az egyszerűbb, egyetlen agentre építő megoldás a robusztusabb.

Érdemes ugyanakkor reálisan látni a korlátokat is. A hosszabb, többlépéses munkamenetekben az agentek még mindig eltévednek, félreértik a szándékot, vagy magabiztosan rossz irányba indulnak; a felügyelet nélküli, végpontig autonóm fejlesztés egyelőre inkább ígéret, mint mindennapi valóság. Így a

következő évek igazi tétje nem az, hogy eltűnik-e az ember a folyamatból, hanem hogy hol húzódik a kötelező emberi ellenőrzés határa - és ki felel azért, amit az agent végül beír a kódbázisba.

A megbízható, hosszú távú autonómia továbbra is korlátozott, és a felelősség az emberé marad. Így a záró gondolat nem az, hogy a programozás eltűnik, hanem hogy áthelyeződik: a chatablaktól a pull requestig az ember szerepe a közvetlen implementációtól a feladat jó meghatározása, a rendszer megértése, az ellenőrzés és a felelősségvállalás felé mozdult. A programozás nem azért változik, mert nem kell technikai tudás, hanem mert ezt a tudást most máshol kell alkalmazni.

X. SZERKESZTŐI AUDIT

Szerkesztői audit

A cikket nyolc szempont szerint értékelem 1-10 skálán. A megrendelés logikája szerint a 8 pont alatti tételeket javítani kell; az alábbi pontszámok már a javított változatot tükrözik, és minden sornál jelzem, mit erősítettem.

Szempont	Pont	Indoklás	Mit erősítettem / megmaradt kockázat
Technikai pontosság	9	A benchmark- és biztonsági adatok elsődleges forrásból; a vendor-számok elkülönítve.	A SWE-bench Verified és Pro különbségét számszerűsítettem, hogy ne keveredjen össze a két mérce.
Gyakorlati érték	9	A 8. fejezet konkrét, kockázat szerinti keretet ad: mit bíz AI-ra és milyen kontrollal.	A három működő mód (tanácsadó, páros, felügyelt agent) explicité vált.
Forrásminőség	8	Túlnyomórészt elsődleges források (METR, Veracode, DORA, Stanford, arXiv).	Megmaradt: néhány vezetői megszólalás másodlagos/marketingforrás; a megjelenés előtt ellenőrizendő.
Érvelés és logikai felépítés	9	Minden fejezet átmenettel kapcsolódik a következőhöz; a fő állítás végig tartja magát.	A benchmark-fejezet (5.) és a minőség-fejezet (6.) szétválasztása élesebb lett.
Olvashatóság	8	Konkrét lead-jelenet, a zsargont magyarázza vagy kerüli.	Megmaradt: a 6. fejezet adatsűrű; a megjelenéskor egy ábra tehermentesítheti (lásd kiegészítő ábrák).
Fogalmi tisztaság	9	A chatbot/copilot/vibe coding/agentic megkülönböztetés következetes.	A vibe coding Karpathy-féle és köznyelvi jelentését explicité elkülönítettem (Willison-elhatárolás).
Hype-mentesség	9	Minden gyorsulási állítás mellett ott az ellenbizonyíték (METR, self-assessment gap).	A munkaerőpiaci állításhoz hozzátettem a dán ellenpéldát és a makrokonfundálókat.
CtrlPlane-hang	8	Gyakorlati, józan, technológiailag pontos, nézőpont nélküli hype nélkül.	Megmaradt: a záró bekezdés kissé esszéisztikus; ha szárazabb hang kell, rövidíthető.

Átlag: 8,6 / 10. Minden szempont eléri a 8 pontot, így kötelező újraírás nincs; a fenti erősítések a szövegben már benne vannak.

Megmaradó kockázatok - a megjelenés előtt ellenőrizendő

- Változékony számok: a benchmark-eredmények (SWE-bench Verified/Pro, Terminal-Bench), az ARR- és értékelési adatok, valamint a heti felhasználói számok gyorsan avulnak; frissítsük őket közvetlenül a publikálás előtt.
- Vezetői idézetek: a konkrét vezetői megszólalásokat és a „kód többségét már az AI írja” típusú állításokat érdemes pontos, elsődleges forráshoz kötni vagy óvatosabban fogalmazni.
- METR-kontextus: a -19% és a későbbi követő mérés számait és a szelekciós torzítást pontosan idézzük, mert könnyen félreolvasható.
- Egyedi esetek: a Replit-adatbázistörlesztés és a VibeScamming illusztráció, nem statisztika; tartsuk meg így a szövegben.

KIEGÉSZÍTŐ ANYAGOK

Kiegészítő anyagok

Newsletter-bevezető (kb. 150 szó)

2022-ben még bemásoltuk a hibás függvényt a ChatGPT-be, elolvastuk a választ, és kézzel javítottunk. Ma egy issue-t adunk át egy agentnek, és kész pull requestet kapunk vissza. Új cikkünk azt járja körbe, mi változott meg valójában a chatablaktól a pull requestig - és miért nem ugyanaz a gyorsabb kód, mint a jobb szoftver. Végigvesszük, hogyan csúszott a munka legkisebb egysége a kódrészletről a feladatra, mit mutatnak valójában a benchmarkok és a kontrollált kísérletek (igen, a METR szerint a tapasztalt fejlesztők lassultak, miközben gyorsulást éltek meg), és hogyan érdemes ma használni ezeket az eszközöket - feladattípus és kockázat szerint. Hype nélkül, számokkal. Olvasd el, mielőtt a következő agentnek átadod a kulcsokat.

Felhasználás: e-mail newsletter bevezető, a teljes cikkre mutató linkkel zárva.

LinkedIn-poszt (kb. 200 szó)

A chatablaktól a pull requestig: szerintem nem azt érdemes kérdezni, hogy az AI gyorsabban ír-e kódot, hanem hogy mi lett a munka legkisebb egysége.

Négy év alatt végimentünk egy íven. A chatbotba bemásoltuk a kódot. A copilot beköltözött az editorba. A vibe codinggal már nem-fejlesztők is szállítottak prototípust. Az agentic codingnál pedig már nem kódrészletet, hanem feladatot adunk át.

És itt jön a két kimondatlan dolog. A METR kontrollált kísérletében a tapasztalt fejlesztők ismerős kódbázison lassultak az AI-tól - miközben azt hitték, gyorsabbak. És a generált kód biztonsági hibaaránya mérhetően magasabb. Vagyis a gyorsabb kód nem feltétlenül jobb szoftver: csökken a kód előállításának költsége, de nő az ellenőrizendő kód és a review-teher.

A fejlesztő szerepe nem eltűnik, hanem áthelyeződik: specifikáció, architektúra, ellenőrzés, felelősség. Ti hol húznátok meg a határt - mit bízok már az agentre, és mit semmiképp? #AgenticCoding #SoftwareEngineering #AI

Felhasználás: személyes vagy Meniva/CtrlPlane LinkedIn-poszt; a hashtagek ízlés szerint cserélhetők.

Négy ábra a cikkhez

Az alábbi négy ábra közvetlenül használható, vagy átadható grafikusnak. Mindegyik a navy/teal arculatot követi.

1. ábra - A négy munkamód összehasonlítása

Dimenzió	Chatbot	Copilot	Vibe coding	Agentic coding
Alapegység	Kérdés	Kódrészlet	Funkció / prototípus	Feladat / issue
Kontextus	Prompt	Aktív fájl / IDE	Projektleírás	Repository és eszközök
Ember szerepe	Kérdező	Közvetlen kódoló	Termékirányító	Feladatadó és reviewer
AI szerepe	Válaszadó	Kiegészítő	Implementáló	Végrehajtó agent
Autonómia	Alacsony	Alacsony	Közepes	Közepes / magas
Fő kockázat	Téves válasz	Hibás kódrészlet	Meg nem értett rendszer	Nagy hatókörű hibás módosítás
Jellemző eszköz	ChatGPT, Rasa	GitHub Copilot	Lovable, Bolt, Cursor	Claude Code, Codex, Devin

A határok folytonosak; a tábla tendenciát rögzít, nem éles falakat.

2. ábra - A workflow előtte és utána (2022 és 2026)

Mozzanat	2022 - chatbot-korszak	2026 - agentic korszak
A munka tárgya	Egy függvény vagy hibarészlet	Egy feladat, issue vagy termékötlet
Hol történik	Külön chatablakban	A repository-ban, a fejlesztő eszközeivel
Ember tevékenysége	Kérdez, másol, beilleszt, javít	Feladatot ad, korlátokat szab, review-z
AI tevékenysége	Szöveget generál	Tervez, módosít, futtat, tesztel, PR-t nyit
Ellenőrzés	A válasz átnézése	Tesztek, diff és pull request review
Fő szűk keresztmetszet	Kontextus és integráció	Review-kapacitás és felelősség

Ugyanaz a fejlesztő, két munkamód: a hangsúly az implementációról az irányításra és ellenőrzésre tolódik.

3. ábra - „Mit bízz az AI-ra?” - kockázat szerinti döntési keret

Kockázat	Feladattípus (példa)	Ajánlott mód	Emberi kontroll
Alacsony	Boilerplate, dokumentáció, egyszerű átalakítás, tesztváz	Felügyelt agent, hosszú póráz	Diff átnézése
Közepes	Több fájlt érintő feature, refaktor, migráció	Páros programozó vagy szűk feladatú agent	Tesztek + kód-review
Magas	Autentikáció, pénzügyi logika, kriptó, production deploy	Tanácsadó / páros; az agent csak előkészít	Kötelező emberi jóváhagyás

Az ábra a 8. fejezet keretét foglalja össze: nem az eszköz, hanem a feladat kockázata dönt.

4. ábra - Az autonómia-spektrum

Autocomplete inline kiegészítés	IDE chat kérdés-válasz a szerkesztőben	AI-native editor multi-file, beépített agent	Terminál-agent futtat, tesztel, iterál	Felhős / autonóm agent háttérben, issue-to-PR
---	--	--	--	---

← Emberi kontroll csökken ··· Autonómia nő →

Balról jobbra nő az AI önállósága és csökken a közvetlen emberi irányítás; a kettő egyensúlya feladatfüggő.

Öt kiemelhető állítás (pull quote)

Közvetlenül használható kiemelések a tördeléshez vagy a poszthoz.

A változás nem az, hogy az AI gyorsabban ír kódot, hanem hogy a munka legkisebb egysége a kódrészletről a feladatra csúszott.

A METR kontrollált kísérletében a tapasztalt fejlesztők lassultak az AI-tól - miközben gyorsulást éltek meg.

A működő demo nem azonos a fenntartható rendszerrel: az autentikáció, a jogosultságkezelés és a monitoring nem opcionális.

A frontvonalbeli modellek képessége közel ért egymáshoz; egyre inkább a harness dönti el az élményt.

A programozás nem azért változik, mert nem kell technikai tudás, hanem mert ezt a tudást most máshol kell alkalmazni.

Hat további cikkötlet

- 1. Az AI-kompatibilis kódbázis.** Hogyan írj CLAUDE.md / AGENTS.md fájlt, amely ténylegesen segíti az agentet - gyors tesztkör, világos struktúra, szűk jogosultságok.
- 2. A review mint szűk keresztmetszet.** Hány agentet tud egy mérnök egyszerre felügyelni és review-zni, és hol marad kötelező az emberi jóváhagyás?
- 3. Benchmark-olvasókönyv.** Mit mér a SWE-bench Verified, a Pro és a Terminal-Bench, mit nem - és miért fontos a kontaminációállóság és a scaffold.
- 4. A technical cliff anatómiája.** Mi történik a vibe-coded prototípus után az autentikáció, a jogosultságkezelés és az adatbázis szintjén?
- 5. Junior pálya az AI korában.** Mit tegyen, aki most kezd - a magyar piacra szabva, az ellentmondó munkaerőpiaci adatok józan értelmezésével.
- 6. Multi-agent vagy egyetlen agent?** Az Anthropic és a Cognition vitája gyakorlati nyelven: mikor éri meg több agentet összehangolni, és mikor nem.